

언리얼 엔진 C++ 환경에서 LLM을 활용한 소규모 게임 개발 효율성 연구

졸업작품 개발 사례를 중심으로

Sung-Wook Cho[○], Min-Woo Ji, Daniel Kim, Dae-Hyun Lee
Tech University of Korea Dept. of Game & Multimedia Engineering



연구 배경

최근 전 세계 IT 업계는 '개발자 채용 감소'와 'AI 인재 수요 급증'이라는 구조적 전환기를 맞이하고 있다. 2021년 이후 개발자 수요는 둔화되었으며, 2022년부터 2024년 사이 전 세 계적으로 약 56만 명 이상의 IT 인력이 감축되었다. 이러한 흐름은 게임 업계에도 직결되어, 신입 개발자의 진입 장벽은 높아지는 반면 LLM을 도구로 활용해 고도의 설계를 수행할 수 있는 능력은 소규모 개발 환경에서 생존을 위한 필수 역량이 되고 있다. 2020년 GPT-3의 등장을 기점으로 시작된 코딩 보조 능력은 2026년 현재 Claude Opus 4.5 및 GPT-5.4와 같은 모델들이 SWE- bench Verified에서 80%, HumanEval에서 95% 이상의 정답률을 기록하며 포화 상태에 이르렀다. 이제 LLM은 단순한 '코드 자동 완성'을 넘어, 개발자의 자연어 의도를 이해하고 복잡한 아키텍처를 설계하는 이른바 '바이트 코딩' 패러다임을 형성하고 있다.

연구 목적

본 연구의 목적은 언리얼 엔진 C++라는 높은 기술적 진입 장벽을 가진 환경에서, LLM 기반의 '바이트 코딩'이 소규모 게임 프로젝트의 생산 한계선을 어떻게 확장시키는지 규명하는 데 있다.

연구 방법

■ 기초 공수(Baseline) 대비 분석

LLM의 보조 없이 진행되었을 때의 예상 소요 시간과 실제 구현 시간을 비교한다.

■ 기술 장벽 극복 사례 연구

개발자의 기존 숙련도만으로는 장시간이 소요되었을 작업을 LLM의 아키텍처 가이드를 통해 해결한 사례를 분석한다.

■ 실제 사례 기반 분석

본 연구팀의 졸업작품 'Containment Cleanup Detail' 개발 과정 중 발생한 구체적인 에피소드 사례를 들어 LLM이 미친 영향을 서술한다.

연구 내용 및 사례

1 Chaos Destruction 기반 캐릭터 파편화 및 액터화 시스템

본 프로젝트는 청소 및 수거, 그리고 이를 방해하는 적대적 개체(SCP)와의 상호작용이 핵심 게임성(Gameplay)을 구성하기에 언리얼 엔진의 Chaos Destruction 시스템을 활용하여 캐릭터 메쉬를 파편화하고, 생성된 각 파편에 개별적인 물리 속성을 부여하여 상호작용 가능한 독립적 액터(Actor)로 변환하는 것을 목표로 하였다.

1단계: Chaos Geometry Collection 생성

해당 단계에서는 LLM의 직접적인 활용보다는 개발자의 수동 작업 비중이 높았다. 언리얼 엔진의 Fracture Mode를 통해 캐릭터 메쉬의 파편화 지점을 지정하고 하위 메쉬를 생성하는 과정은 시각적 판단이 수반되는 작업이기 때문이다. 이 공정을 통해 물리 기반 파편으로 치환될 기초 데이터를 확보하였다.

2단계: 파편의 액터화 및 물리 속성 부여

파편화된 Geometry Collection 데이터 자체만으로는 독립적인 상호작용 및 세부 물리 효과 적용에 한계가 존재한다. 따라서 상기 단계에서 생성된 파편 메쉬를 독립적인 액터로 전환하는 공정을 수행하였다. 이 과정에서 LLM은 스폰할 파편 메쉬 리스트의 동적 생성 로직과 SkeletalMeshComponent 기반의 액터 클래스 구조 설계에 활용되었다. 특히 시체 파편에 Ragdoll 효과를 부여하기 위한 LLM의 기술적 제언은 실증적으로 유효함을 확인하였다. 물리 효과의 정밀도를 높이기 위한 Physics Asset 생성 및 Skin Weight 조정은 개발자가 직접 수행하였으나, 이를 제어하는 시스템 코드 작성 영역에서는 LLM이 결정적인 기여를 하였다.

개발 시간 단축 및 학습 비용 절감

LLM은 숙련되지 않은 개발자가 초기 아키텍처를 설계하는 기존 방식 대비 약 70% 이상의 시간 효율을 달성하였으며, 이는 검색 및 시행착오에 소요되는 시간을 실제 구현 시간으로 전환했음을 의미한다.



2 LLM 멀티모달 기능을 활용한 시각적 로직의 코드 변환

본 연구에서는 소규모 게임 개발 과정에서 성능 최적화와 개발 속도 사이의 간극을 줄이기 위해, LLM의 이미지 인식 기능을 활용한 노드 분석 및 C++ 변환 프로세스를 시도하였다.

시각적 노드 그래프의 논리적 인식 및 구조화

전통적인 방식에서 블루프린트 로직을 C++로 이식하기 위해서는 개발자가 직접 노드의 실행 흐름과 데이터 핀의 연결 관계를 역설계해야 했다. 본 연구에서는 멀티모달 LLM에 블루프린트 에디터의 스크린샷을 입력값으로 제공하였다.

C++ 클래스 변환

인식된 시각적 논리를 바탕으로, LLM은 언리얼 엔진의 코딩 표준을 준수하는 C++ 코드를 생성하는데 성공하였다. LLM은 블루프린트의 각 노드에 대응하는 UFUNCTION 및 UPROPERTY 매크로를 포함한 헤더(.h)와 소스(.cpp) 파일을 구성하였다. 특히 가상 머신 오버헤드가 발생하는 블루프린트 로직을 네이티브 C++ 코드로 치환함으로써 실행 성능을 극대화하였다. 또한 생성된 코드에서 발생할 수 있는 잠재적인 널 포인터 참조나 타입 불일치를 사전에 식별하고 보정하는 검수 과정을 수행하였다.

'프로토타이핑-최적화' 워크플로우의 선순환

멀티모달 기능을 활용한 변환 프로세스는 단순한 도구의 활용을 넘어, 개발 주기 전체의 효율성을 개선하였다. 시각적 직관성이 높은 블루프린트로 기능을 빠르게 구현(Prototyping)한 후, LLM을 통해 즉시 C++로 안정화(Stabilization)하는 이분법적 접근이 가능해졌으며, 복잡한 로직을 텍스트로 설명하거나 문서화할 필요 없이 스크린샷 한 장으로 의도를 전달함으로써, 개발자와 AI 간의 정보 전달 효율을 극대화하고 개발 흐름이 끊기는 맥락 전환 비용을 최소화할 수 있었다.

한계점 및 결론

외부 SDK 및 복합 네트워크 환경의 디버깅 한계

본 프로젝트에서 도입한 스팀 온라인 세션(Steam Online Session) 구현 과정에서 LLM은 명확한 해결책을 제시하는 데 한계를 보였다.

기술적 할루시네이션과 버전 파편화 문제

개발 과정에서 LLM은 명시적인 버전 지정에도 불구하고 잘못된 정보를 제공하는 할루시네이션 현상을 보였다. 프롬프트에 'Unreal Engine 5.7'를 명시하였음에도, 최신 API와의 호환성이 결여된 구 버전(UE4)의 함수를 제안하는 등의 사례가 빈번히 발생하였다. 이에 개발자가 컴파일 에러를 직접 수정하기 위해 다시 공식 문서를 교차 검증해야 하는 '역(逆) 효율성'이 발생하였다.

대화의 관성과 문맥 오염 (Context Contamination)

LLM과의 장기적인 협업 과정에서 발생하는 '정보 고착화' 현상은 개발의 유연성을 떨어뜨리는 또 다른 한계점으로 나타났다. 대화가 길어질수록 LLM은 이전 발화(Utterance)에서 형성된 맥락에 과도하게 의존하는 경향을 보였다. 이로 인해 개발자가 명확히 새로운 기준이나 수정된 라이브러리 정보를 제공하더라도, 모델이 이전 답변의 코드 구조를 반복 출력하거나 수정 사항을 누락하는 '대화의 관성' 현상이 발생하여 불필요하게 반복해서 수정사항을 지시해야 했고, 때때로 대화 기록을 초기화하고 새로운 세션을 시작해야 하는 상황이 빈번하는 등 재입력 비용이 필요했다.

결론

위와 같은 한계점들은 LLM 기반 개발 환경에서 개발자의 역할이 '작성'에서 '검토 및 검증'으로 변화해야 함을 시사한다. 개발자 개인의 도메인 지식(Domain Knowledge)이 전제되어야만 진정한 효율성 향상을 기대할 수 있으며, 아직 LLM은 개발자를 완전히 대체할 수 없고, 개발자의 역량에 따라 그 활용 가치가 결정되는 고도의 '지능형 도구'로서 기능함을 확인하였다.